Numerical Algorithms (2022) 89:529–549
https://doi.org/10.1007/s11075-021-01124-7

ORIGINAL PAPER

On the unitary block-decomposability of 1-parameter matrix flows and static matrices

Frank Uhlig[1]

Received: 19 November 2020 / Accepted: 27 April 2021 / Published online: 20 May 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

For general complex or real 1-parameter matrix flows $A(t)_{n,n}$ and for static matrices $A \in \mathbb{C}_{n,n}$ alike, this paper considers ways to decompose matrix flows and single matrices globally via one constant matrix similarity $C_{n,n}$ as $A(t) = C^{-1} \cdot \text{diag}(A_1(t), ..., A_\ell(t)) \cdot C$ or $A = C^{-1} \cdot \text{diag}(A_1, ..., A_\ell) \cdot C$ with each diagonal block $A_k(t)$ or A_k square and their number ℓ exceeding 1 if this is possible. The theory behind our proposed algorithm is elementary and uses the concept of invariant subspaces for the MATLAB `eig` computed 'eigenvectors' of one associated flow matrix $B(t_a)$ to find the coarsest simultaneous block structure for all flow matrices $B(t_b)$. The method works efficiently for all time-varying matrix flows $A(t)$, be they real or complex, normal, with Jordan structures or repeated eigenvalues, differentiable, continuous, or discontinuous in t, and likewise for all fixed entry matrices A. Our intended aim is to discover unitarily diagonal-block decomposable flows as they originate in real-time from sensor given data for time-varying matrix problems that are unitarily invariant. Then, the complexities of their numerical treatments decrease by adopting 'divide and conquer' methods for their diagonal blocks. In the process, we discover and study k-normal fixed entry matrix classes that can be decomposed under unitary similarities into various k-dimensional block-diagonal forms.

Keywords Time-varying matrices · Matrix flow · Unitary matrix similarity · Block-diagonalizable matrix · Decomposable matrix flow · k-normal matrix · Quantum physics · Numerical matrix algorithm

AMS Subject Classification 15A99 · 15A21 · 81Q10 · 15B99 · 65F99

✉ Frank Uhlig
uhligfd@auburn.edu

[1] Department of Mathematics and Statistics, Auburn University, Auburn, AL 36849, USA

Preface

Matrix block decompositions have been studied for almost a century. They appeared first in the early days of quantum physics in the 1920s and were essential to comprehend how higher atomic weight elements might have been formed from lighter ones in the primeval stages of the universe under aggregation, pressure, and heat, see, e.g. [12]. Related eigencurve crossing processes were modeled by 1-parameter hermitean matrix flows $A(t)$ and their parameter-varying eigencurves were found to never cross for indecomposable hermitean matrix flows. This eventually led matrix theoreticians and numericalists to study eigencurve crossings in order to grasp the notion of unitarily decomposable hermitean matrix flows. These studies have been conducted from the fixed entry, static matrix theoretical and matrix numerics standpoints via backward stable methods. They use matrix similarities, matrix factorizations, basic linear algebra subroutines (BLAs), and so forth; all in a Wilkinsonian way [22], paying attention to error analyses, to perturbation effects, and so forth. Eigencurve crossings have been found and algorithms proposed to compute them, yet with little effect on actually decomposing decomposable hermitean or general complex matrix flows or static complex or real matrices, nor how to tell whether a given flow or static matrix is unitarily or orthogonally decomposable or not by one fixed matrix similarity.

This author has contributed to the eigencrossings literature in [19] but was ultimately not satisfied with his results. Eventually, I understood that eigencurve studies are looking for symptoms of the unitary similarity decomposability of a given matrix or matrix flow, but they do not address the underlying problem of possible block-diagonalizability. This led me to approach the matrix decomposability problem more simply via invariant eigenspace theory for hermitean matrix flows $A(t)$. The resulting numerical matrix block-decomposing algorithm ($\mathcal{MBDA}$) of this paper uses one hermitean matrix eigenanalysis and relies on matrix entry magnitude spreads and logic 0–1 matrix structural computations to establish a block diagonal version of any given decomposable matrix or matrix flow with the same unitarily invariant qualities as the originally given dense matrix or matrix flow.

The theoretical method is new, it solves a previously intractable matrix quandary and it does so elegantly in theory. In practice, it still has certain limitations brought on by the qualities of the necessarily larger magnitude matrix entry spreads for decomposed matrices. It has practical applications for many unitarily invariant matrix property computations. Besides, this method, combining new aspects of matrix flow computations with static matrix numerics lies outside our current knowledge base of matrix theory and its applications. There is, for example and unfortunately, little knowledge or experience available for matrix flows on how to overcome such seemingly intrinsic numerical limitations of our theory based algorithm.

This is a new territory and both, fixed entry matrices A and general or hermitean matrix flows $A(t)$ can now be block-decomposed by unitary similarities very simply—if that is possible for a given matrix problem—and new insights can be gained for static matrices and matrix flows alike. This helps to compute many aspects of unitarily decomposable matrices in a 'divide and conquer' fashion much more speedily, such as—for example—drawing the field of values of decomposing matrices successfully via the Zeroing Neural Network path following method in [21].

1 Introduction

This paper studies time-varying, i.e., 1-parameter varying matrix flows $A(t) \in \mathbb{C}_{n,n}$ over an interval $t_o \leq t \leq t_f \in \mathbb{R}$ or when t follows a finite section of a curve in $\mathbb{C}$. Our approach connects matrix flows with fixed entry matrices $A \in \mathbb{C}_{n,n}$. In many applications and in many matrix computations, it is advantageous to know whether a dense matrix flow $A(t)$ or a dense general static matrix A can be decomposed into an array of diagonal blocks $A_k(t)$ or A_k (for $k = 1, ..., \ell$) of smaller dimensions, i.e.

$$A(t)_{n,n} = C^{-1} \cdot \begin{pmatrix} A_1(t) & O & O & \cdots & O \\ O & A_2(t) & O & \cdots & O \\ O & O & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & O \\ O & O & \cdots & O & A_\ell(t) \end{pmatrix} \cdot C \text{ or } A_{n,n} = C^{-1} \cdot \text{diag}(A_1, ..., A_\ell) \cdot C. \tag{1}$$

Here, $C \in \mathbb{C}_{n,n}$ is an invertible fixed entry matrix that remains constant for all flow parameters t. If a matrix flow $A(t)$ or a static matrix A can be decomposed in this fashion with $\ell > 1$, then many numerical problems for $A(t)$ and A may be 'divided and conquered' into ℓ smaller subproblems for the individual blocks $A_k(t)$ or A_k and these subproblems can usually be solved more quickly. The matrix flow $A(t)$ may derive over time by discretizing given equations or it may be generated from sensor data that arrives at a constant discrete sampling rate τ such as $\tau = 0.02$ s or 50 Hz.

Decomposable matrix flows have been intimately linked to eigencurve crossings of matrix flows $A(t)$ for over 90 years since their original quantum physics connection [12] was discovered. In 1927 and 1929, Hund [10] and von Neumann and Wigner [15] proved that single parameter hermitean flows $A(t) = (A(t))^*$ whose eigencurves cross each other must be decomposable via a fixed unitary matrix C in the sense of (1). Moreover [15] showed that 'hyperbolic avoidance' of flow eigencurves implies that the associated eigenvalues belong to the same diagonal block of a unitarily decomposable hermitean matrix flow. An eigencurve crossing is sufficient for hermitean matrix flow decomposability, but the converse implication is not true, i.e. eigencurve analyses can exhibit block-diagonalizability symptoms but they are not sufficient to establish unitary block-diagonalizability. It may be well worth the effort now to unearth the original quantum physics models and their associated hermitean flow matrices for the creation of higher atomic order elements and study their unitary decomposability directly rather than look at symptoms. In [19], the author has studied the eigencurves of hermitean and general matrix flows and developed an algorithm to deduct the coarsest block-diagonalization dimensions of hermitean matrix flows from their eigencurve crossing data. The biggest drawback of that method for deciding matrix or data decomposability is the fact that hermitean and general matrix flows $A(t) \in \mathbb{C}_{n,n}$ need not show eigencurve crossings at all, even if they are decomposable. This paper introduces a different algorithm that uses elementary matrix invariant subspace theory to decompose matrix flows and static matrices

unitarily into block-diagonal form expediently and accurately—if at all possible—or it establishes that such decompositions are impossible for $A(t)$ or $A_{n,n}$. The fundamental matrix flow results are then applied to the static matrix decomposability question for general matrices, to k-normal matrices, as well as to several decomposable static test matrices from MATLAB's 'gallery' in Section 3 and the Applications Section 4. We also review some adjacent matrix block-decomposition efforts in the literature, such as the work [1, 14].

Details, numerical codes and tests follow.

2 Theory

This section introduces the elementary theory of decomposable matrix flows $A(t) \in \mathbb{C}_{n,n}$ and shows how to unitarily decompose general static matrices $A \in \mathbb{C}_{n,n}$ into block-diagonal form if that is possible.

To start, we consider a 'proper' n by n hermitean time-varying matrix flow $A(t) \in \mathbb{C}_{n,n}$ that can be block-diagonalized uniformly as described in (1), using a fixed unitary matrix similarity for all $t_o \leq t \leq t_f$. Let us assume that we have $\ell > 1$ diagonal blocks here for 'properness'. A square matrix flow $A(t)$ of hermitean matrices allows us to diagonalize flow matrices $A(t_a)$ with $t_a \in [t_o, t_f]$ via a unitary similarity transformation $V(t_a)$ so that $A(t_a) \cdot V(t_a) = V(t_a) \cdot D(t_a)$ and $D(t_a)$ is real diagonal. The transforming unitary matrix $V(t_a)$ contains the eigenvectors of $A(t_a)$ in its columns and the eigenvalues of $A(t_a)$ appear on the diagonal of $D(t_a)$ if we use MATLAB's built-in `eig` function for example. Assuming that $A(t_a) = C^{-1} \cdot \text{blockdiag}(A_1(t_a), ..., A_\ell(t_a)) \cdot C$ for some nonsingular fixed entry matrix C, each eigenvector in $V(t_a)$ is associated with one of the eigenvalues of $A(t_a)$ and in fact the eigenvector columns of $V(t_a)$ that are associated with the eigenvalues of one diagonal block $A_i(t_a)$ form an orthonormal basis for an invariant subspace of $A(t)$ of which there are $\ell > 1$ by assumption.

All matrices of the decomposable hermitean flow $A(t)$ share the same invariant subspace structure as expressed in (1). Therefore, for any $t_b \neq t_a \in [t_o, t_f]$, the matrix $A(t_b)$ must be block diagonalizable conformably with the block structure of $A(t)$ as indicated by the $A_i(t_a)$ blocks. And thus

$$\tilde{A}_{t_a}(t_b) = (\tilde{V}(t_a))^* \cdot A(t_b) \cdot \tilde{V}(t_a) \tag{2}$$

will be block-diagonal with the same common block structure as soon as we have re-arranged the eigenvector rows or columns of $V(t_a) = V(t_a)^*$ in $\tilde{V}(t_a)$ into ℓ groups that generate equal zero and non-zero pattern columns in $\tilde{A}_{t_a}(t_b)$. The rearrangement of the columns of $V(t_a)$ can be achieved numerically when the spread of entry magnitudes in $\tilde{A}_{t_a}(t_b)$ has increased sufficiently by looking at the logic 0-1 `spy` matrix $A_{logic}(t_b)$ of $\tilde{A}_{t_a}(t_b)$ for a certain threshold in MATLAB. The actual setting of thresholds is described and tested in detail in Section 3. When we re-sort the columns of $V(t_a)$ so that logic 0-1 row or column vectors in $A_{logic}(t_b)$ (and thus of $\tilde{A}_{t_a}(t_b)$) fall into ℓ distinct groups according to the location of their almost-zero and well above

zero magnitude entries, we obtain $\tilde{V} = \tilde{V}(t_a)$ which by (1) block-diagonalizes every flow matrix $A(t)$.

Theorem 1 *If for all t, a hermitean time-varying matrix flow $A(t)$ can be properly block-diagonalized by a constant unitary similarity $U^*...\ U$ into $l > 1$ blocks, then the unitary eigenvector matrix V of any flow matrix $A(t_a)$ can be re-arranged column-wise in $\tilde{V}$ so that any matrix $\tilde{V}^* A(t_b)\tilde{V}$ with $t_b \neq t_a$ has the identical or a finer block-diagonal structure for all t_b. And vice versa, if the unitary eigenvector matrix V of one hermitean matrix flow matrix $A(t_a)$ creates a block-diagonalizable logic 0-1 pattern matrix for $A(t_b)$ with $t_b \neq t_a$ under a similarity by a column-rearranged version $\tilde{V}$ of V, then all matrices in the hermitean flow $A(t)$ are simultaneously and uniformly block-diagonalizable by the same unitary similarity $\tilde{V}^*\ A(..)\ \tilde{V}$ as $A(t_b)$.*

For general complex matrix flows $A(t)_{n,n}$ that are diagonalizable throughout their single-parameter range, the same invariant subspace argument holds except that the unitary eigenvector matrix $\tilde{V}(t_a)$ similarity needs to be replaced by a general similarity via a nonsingular matrix $\tilde{W}(t_a)$ so that the inverse $\tilde{V}(t_a)^*$ of $\tilde{V}(t_a)$ becomes $\tilde{W}(t_a)^{-1}$ in formula (2).

Theorem 2 *A diagonalizable general complex time-varying matrix flow $A(t)$ can be properly and uniformly block-diagonalized by a constant matrix similarity $C^{-1}...\ C$ into $\ell > 1$ blocks if and only if the eigenvectors of any flow matrix $A(t_a)$ will block-diagonalize – upon re-sorting – every other flow matrix $A(t_b)$ by similarity into block-diagonal form which may be finer than the coarsest possible block-diagonal form of that flow.*

Here, the term 'coarsest block-diagonal form' refers to one with the minimal possible block number $\ell > 1$ in (1). Note for example, that $D(t_a)$ of formula (2) represents the finest, i.e. a 1 by 1 block-diagonalization with $\ell = n$ for $A(t_a) \in \mathbb{C}_{n,n}$.

Unitary block-decompositions of static complex matrices $A \in \mathbb{C}_{n,n}$ can be computed by using Theorems 1 and 2 in reverse on the related hermitean matrix flow $\mathcal{F}_A(t) = \cos(t)H + \sin(t)K$ that is generated from the hermitean and skew parts $H = (A + A^*)/2$ and $K = (A - A^*)/(2i)$ of A. This idea was inspired by its use in field of values boundary computations, see [11] and [21].

To find a unitary block-diagonalization of any square matrix $A \in \mathbb{C}_{n,n}$ with our elementary block-diagonalising method requires us to compute the eigenvectors via the QR algorithm of one hermitean matrix $\mathcal{F}_A(t_a)$. This can be done at much smaller hermitean $O(n^3)$ QR cost than a general matrix QR eigenanalysis on A itself. Thereafter, we just need to check the 0–1 pattern matrix of any $\mathcal{F}_A(t_b)$ where $t_b \neq t_a$. If its logic 0–1 pattern shows a block diagonalizable pattern, we re-sort the eigenvector columns for $\mathcal{F}_A(t_a)$ that reside in V into equal row pattern groups in $\tilde{V}$. Then the matrix $\tilde{A} = (\tilde{V})^* \cdot A \cdot \tilde{V}$ will be block diagonal per Theorems 1 and 2. $\tilde{A}$'s eigenvalues can then be found more expediently from the smaller general diagonal blocks of $\tilde{A}$ than using MATLAB's `eig` function on the given dense, but unitarily decomposable n by n matrix A.

Theorem 3 *A diagonalizable general complex matrix A can be properly block-diagonalized by a constant unitary matrix similarity $C^*\ A\ C$ into $l > 1$ blocks if and only if the eigenvectors of any associated hermitean flow matrix $\mathcal{F}_A(t_a) = \cos(t_a)H + \sin(t_a)K$ with $H = (A + A^*)/2$ and $K = (A - A^*)/(2i)$ will block-diagonalize – upon re-sorting – every other flow matrix $\mathcal{F}_A(t_b)$ by unitary similarity into block-diagonal form.*

Proof If $A \in \mathbb{C}_{n,n}$ can be properly block-decomposed by a unitary similarity C into $\ell > 1$ diagonal blocks then both $H = (A + A^*)/2$ and $K = (A - A^*)/(2i)$ similarly block-decompose under the unitary similarity by C. Hence $\mathcal{F}_A(t)$ block-decomposes via $C^*...C$ in the same way as A.
Conversely, if the unitary eigenvector matrix V of one hermitean flow matrix $\mathcal{F}_A(t_a) = \cos(t_a)H + \sin(t_a)K$ block-diagonalizes any other flow matrix $\mathcal{F}_A(t_b)$ by unitary similarity into proper block-diagonal form, then by Theorem 1 $\mathcal{F}_A(0) = H$ and $\mathcal{F}_A(\pi/2) = K$ are likewise unitarily block-diagonalized by $V^*...V$ and hence $A = H + iK$ is also. □

The next section deals with computing the minimal number ℓ of invariant subspaces of a properly decomposable matrix flow $A(t)$ or a complex matrix A by re-sorting the columns of their respective eigenvector matrices $V(t_a)$ or $W(t_a)$, so that the coarsest simultaneous diagonal block reduction (or a finer one) can be achieved for any matrix flow $A(t)$ or any static matrix A effectively—provided that ℓ is found to exceed 1.

3 The numerical block-diagonalization algorithm and computed results

In order to develop and test our matrix block-diagonalization algorithms, we generally need to construct dense unitarily block-diagonalizable test examples first since most known unitarily block-diagonalizable matrices and matrix flows are normal ones. For this purpose, we start from a block-diagonal matrix $B \in \mathbb{C}_{n,n}$ or a 1-parameter block-diagonal n by n matrix flow $B(t)$ and construct a randomized unitary matrix $Q \in \mathbb{C}_{n,n}$ by using the MATLAB command `>> [Q,~]= qr(randn(n)+1i*randn(n))` for example. Then, $A = Q^*BQ$ and $A(t) = Q^*B(t)Q$ generally become dense block-diagonalizable matrices or matrix flows that are suitable for testing our algorithms on the dense versions A or $A(t)$ of B or $B(t)$.

As theory tells us for matrix flows, to solve the matrix flow decomposability problem for diagonalizable matrix flows, it suffices to compute

(I) the eigenvector matrix $X(t_a)$ of one flow matrix $A(t_a)$ and apply the similarity $(X(t_a))^{-1} \cdot\ A(t_b)\ \cdot X(t_a)$ to any other flow matrix $A(t_b)$.

Then theoretically, all entries outside of the diagonal blocks of the flow matrix $(X(t_a))^{-1} \cdot A(t_b) \cdot X(t_a)$ with $t_b \neq t_a$ must be zero as $A(t)$ is block-diagonalizable

by construction. But numerical rounding errors and conditioning problems will generally create relatively small magnitude entries in some or all entry positions of $\hat{A}(t_b) = (X(t_a))^{-1} \cdot A(t_b) \cdot X(t_a)$ outside its yet unknown diagonal block structure. Therefore, we must use suitable entry size thresholds before establishing block-diagonalizability.

This section treats the block-diagonalization problem in three separate parts. Section 3.1 deals with hermitean matrix flows, Section 3.2 treats general matrix flows and Section 3.3 applies these results to finding unitary unitary block-diagonalizations for static matrices, if they exist.

3.1 Unitarily decomposable hermitean matrix flows

Assuming that all matrices $A(t_a) = (A(t)_a)^*$ of a given block-decomposable hermitean flow $A(t)$ are dense, the overall spread of entry magnitudes in $\hat{A}(t_b) = (X(t_a))^* \cdot A(t_b) \cdot X(t_a)$ must increase from that of $A(t_a)$ and $A(t_b)$ since the original dense flow $A(t)$ will contain very few small magnitude entries while the numerically block-diagonalized matrix $\hat{A}(t_b)$ will contain whole off-diagonal blocks of rounding error sized tiny magnitude entries. If the lowest entry magnitude entry in $\hat{A}(t_b)$ is less than our experimentally chosen threshold $\|At_b)\|10^{-13}$, we accept that the flow is unitarily block-diagonalizable and we determine its individual diagonal block sizes via a MATLAB `spy` plot. We do this by collecting equal 0–1 rows of $\hat{A}(t_b)$'s `spy` output into groups and reordering the unitary eigenvectors of $A(t_a)$ in $X(t_a)$ accordingly. Thereafter we express the given flow in its unitarily achieved block diagonal form and exhibit the block structure of $\hat{A}(t_b)$ properly from the threshold defined block dimensions.

I.e. we evaluate

(**II**) the logic 0-1 matrix of the computed matrix $\hat{A}(t_b)$ and then

(**III**) we rearrange its rows and columns by collecting equal 0-1 pattern row vectors therein into groups and compute the corresponding block-diagonal form of $\hat{A}(t_b)$.

This process is a great improvement over what could be gleamed geometrically from eigencurve crossings in [19].

Figure 1 shows nine MATLAB `spy` graphs for a dense block-decomposable hermitean 17 by 17 matrix flow $A(t)$. Reading the images row by row, the first row of graphs shows the MATLAB `spy` 0–1 pattern transitions from $A(t_a)$ to $A(t_b)$; the second row shows the 0–1 pattern transitions from $A(t_b)$ to $A(t_c)$, and the third row shows the pattern transitions from $A(t_c)$ to $A(t_{rd})$ for a randomly chosen parameter $t_{rd} \in \mathbb{C}$.

Column **(A)** in Fig. 1 shows the 0–1 pattern of the diagonalization $D(t_{...})$ of $A(t_{...})$ as obtained via MATLAB's `eig` function. Column **(B)** displays the 0-1 unitary similarity patterns from varying starting matrices $A(t_a)$, $A(t_b)$, $A(t_c)$. Note that the spy graphs in column **(B)** all visually hint at a 7, 4, 3, 2, and 1 block-diagonalization for this flow. The third column **(C)** of `spy` graphs is computed according to the 0–1 data in column **(B)** by collecting the diagonal blocks of the flow matrices $A(t_b)$, $A(t_c)$ and $A(t_{rd})$, respectively, in their proper grouping arrangements row by row. In **(C)**

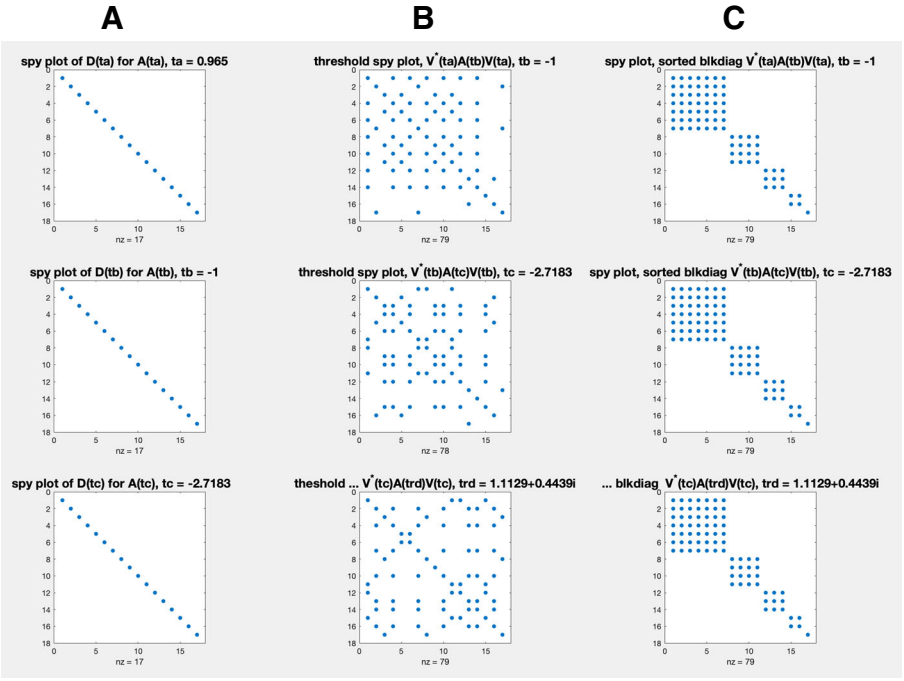


Fig. 1 Hermitean 17-dimensional flow $A(t)$ with a chain progression from $t = t_a$ to t_b to t_c, and to random $t = t_{rd} \in \mathbb{C}$

the same non-zero diagonal blocks of dimensions 7, 4, 3, 2, and 1 appear arranged in decreasing sizes for $A(t_b)$, $A(t_c)$, and $A(t_{rd})$.

The hermitean matrix flow $A(t)_{17,17}$ of this test example was built from a block-diagonal hermitean matrix flow $B(t)$ with blocks of dimensions 7 by 7, 4 by 4, 3 by 3, 2 by 2, and 1 by 1. $B(t)$ was then transformed into the dense general flow $A(t) \in \mathbb{C}_{17,17}$ by a fixed random entry dense and unitary similarity. The spy graph sequences of Fig. 1 were computed by our MATLAB algorithm `dechermflow9.m` from the subfolder *hermitean flows* of [20].

A separate algorithm called `dechermflow9a.m` in the subfolder *hermitean flows* of [20] starts each row of plots with the `spy` plot of the diagonal eigenvalue matrix $D(t_a)$ for $A(t_a)$ and uses $A(t_a)$'s unitary eigenvector matrix V to determine the threshold 0-1 `spy` matrices for $V^*A(t_b)V$, $V^*A(t_c)V$ and $V^*A(t_{rd})V$ in turn. In the second column (**B**) of Fig. 2 the `spy` plots all have identical 0-1 patterns with block-diagonal dimensions 4, 3, 3, 2, and 2 in this 14-dimensional hermitean example flow, as do the re-sorted block-diagonal versions of $V^*A(t_b)V$, $V^*A(t_c)V$ and $V^*A(t_{rd})V$ in plot column (**C**) below.

In [19], a geometric path following Zhang Neural Network method was introduced that applies a path following Zhang Neural Network to the matrix flow block-diagnalization problem. It plots the eigencurves of hermitean matrix flows over a given interval and determines a flow's block-decomposability from the encountered

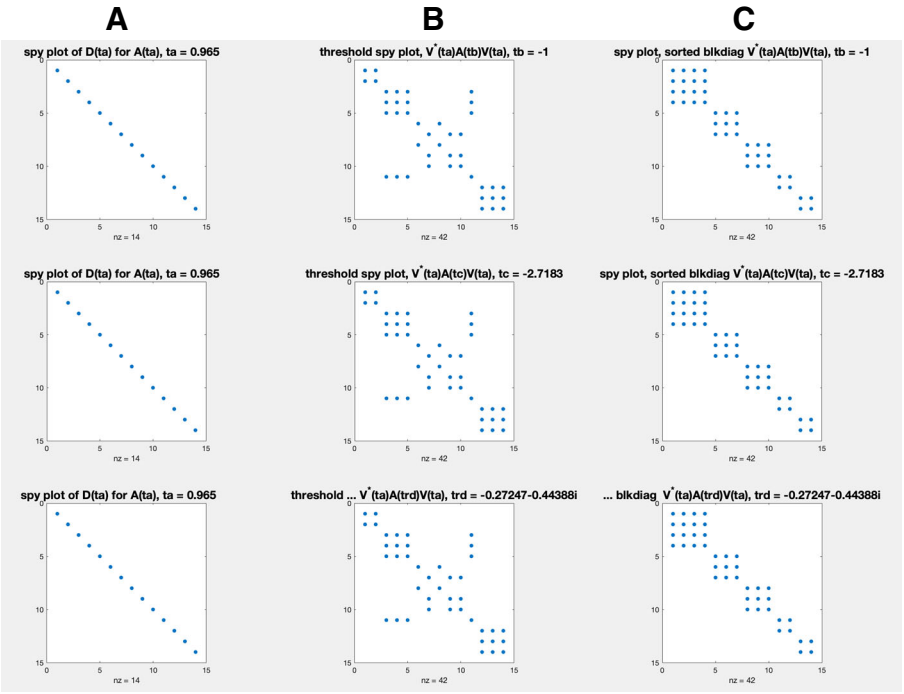


Fig. 2 Hermitean flow $A(t) \in \mathbb{C}_{14,14}$ with three progressions, all starting at $t = t_a$ and going to t_b, to t_c, and to a random $t = t_{rd} \in \mathbb{C}$ in turn from $A(t_a)$

eigencurve crossings and their hyperbolic crossing avoidances numerically. This follows the original ideas of von Neumann and Wigner [15] from 1929. The algorithm in [19] tries to computes the block dimensions of hermitean matrix flows that decompose under unitary similarities. To succeed the chosen interval needs to be sufficiently large.

Figure 3 shows the eigencurve plot for a 11 by 11 hermitean matrix flow $A(t)$ that allows us to ascertain that the flow is unifomly block-diagonalizable into blocks of sizes 7 and 4 by one fixed unitary similarity.

Drawing this plot and finding $A(t)$'s block structure takes around 3.9 s on a Macbook Pro from 2019. Note that the eigencurve von Neumann and Wigner based ZNN algorithm is not capable or designed to compute the actual block-diagonal form of decomposable hermitean matrix flows. The algorithm of this paper computes the actual block-diagonal representation for the block-diagonalizable 11 by 11 test matrix flow $A(t)$ in 0.045 s.

3.2 Block-diagonalizations of general complex matrix flows via standard similarities

A slight adaptation of our algorithm to account for basic matrix similarities $\hat{A}(t_b) = (X(t_a))^{-1} A(t_b) X(t_a)$ instead of unitary ones such as $\hat{A}(t_b) = (V(t_a))^* \cdot A(t_b) \cdot X(t_a)$

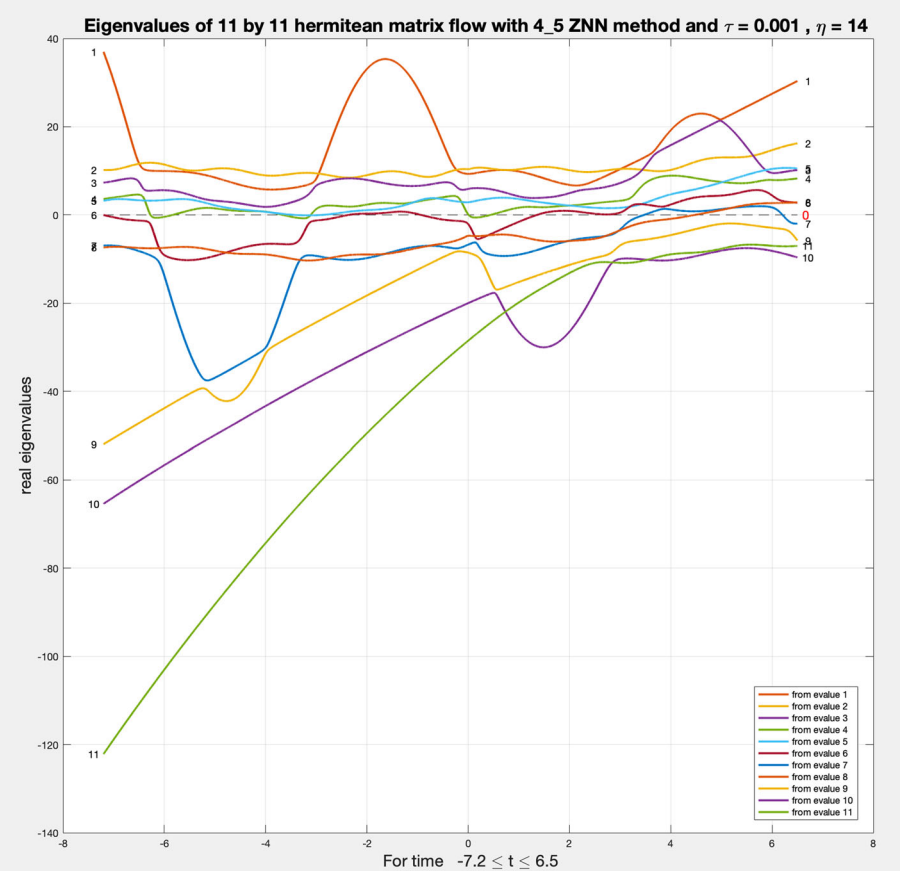


Fig. 3 Eigencurves of a hermitean matrix flow $A(t) \in \mathbb{C}_{11,11}$ that decomposes computationally into two diagonal blocks of sizes 7 and 4

for hermitean matrix flows in Section 3.1 works well here for general matrix flows that are built—for example—from Jordan blocks such as the next 9 by 9 general complex matrix flow example $A(t)$ shows with two Jordan blocks of sizes 4 and 5 in Fig. 4 below. Note that there are fewer above threshold entries in the threshold 0–1 `spy` plots of column **(B)** in Fig. 4, namely just 29 in each row position instead of the expected $4^2 + 5^2 = 42$. Recall that the second plot in row two of Fig. 1 for hermitean flows had just missed one low magnitude entry due to our threshold choice. In both `dechermflow9.m` and `decgenflow910.m` in the subfolder *general flows* of [20] we use the same threshold values of $\|A(t..) \cdot 10^{-13}$ for threshold `spy` plots. But general flows with Jordan block structures behave quite differently as we shall soon learn. Finally, the correct number of above threshold entries in the block-diagonal representation of this test plot is depicted with descending block size orders in each sorted plot of column (**C**) of Fig. 4 which depict the sorted block-diagonal `spy` graphs of $-\hat{A}(t_{...})$.

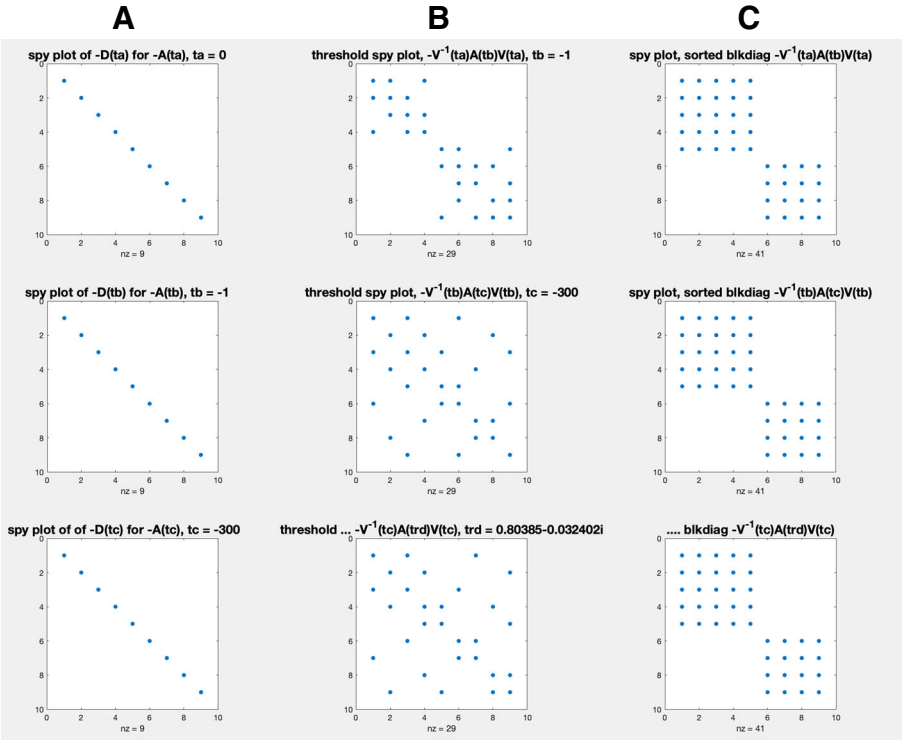


Fig. 4 Dense decomposable complex matrix flow – $A(t) \in \mathbb{C}_{9,9}$, formed by two Jordan blocks of sizes 4 and 5

The presence of Jordan blocks in a general complex matrix flow $A(t)$ and their treatment in `eig.m` may create bands of 0–1 entry rows of all ones when computing $\hat{A}(t_b) = (X(t_a))^{-1} A(t_b) X(t_a)$. These all-ones rows need to be taken care of quite differently in the general matrix flow case than in the hermitean matrix flow case, where such cannot occur.

In plot column **(B)**, Fig. 5 below shows such a banded 0–1 pattern matrix with several all-ones rows for a dense general example flow $A(t)_{14,14}$ that was built from a general complex 14 by 14 matrix flow that contains two Jordan blocks of size 2 and other blocks of dimensions 1 (3-fold), 3, and 4. Note that grouping identical 0-1 rows of column (**B**) together into one diagonal block in column (**C**) for Jordan block containing general flows—as is sufficient for hermitean flows—would result in all rows of the 0–1 pattern `spy` matrix becoming indistinguishable here, indicating falsely that the given complex flow is indecomposable.

Note finally that in the intermediate second column the graphical 0-1 output (**B**) in Figs. 1, 2 and 4–5 may differ from row to row, but that the final graphs in column (**C**) are identical in each test run and correctly sorted by decreasing block sizes.

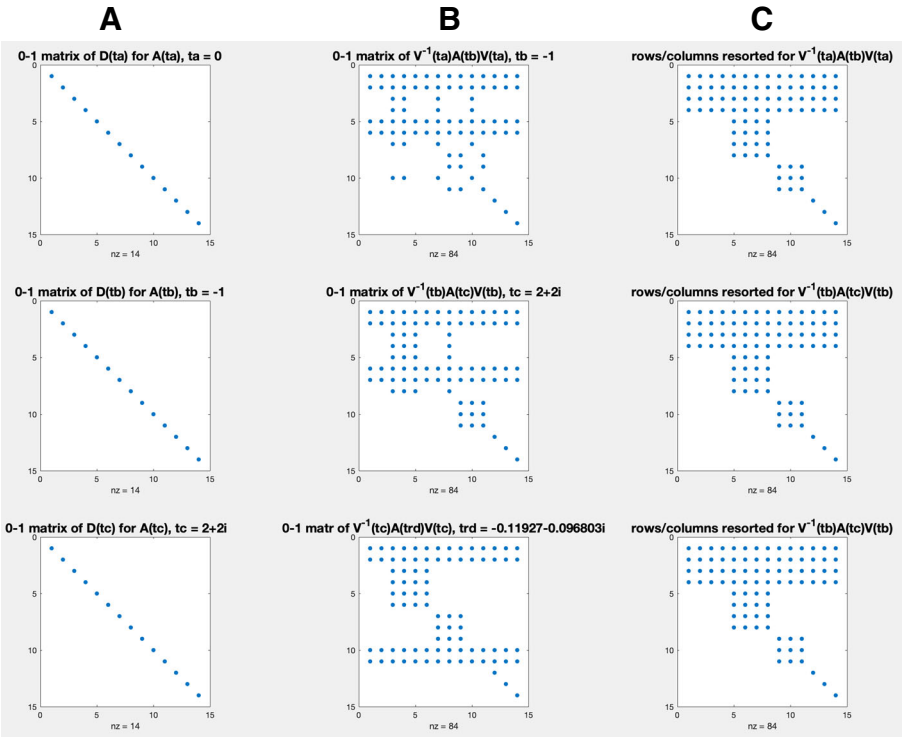


Fig. 5 General decomposable complex matrix flow $A(t)$ with two 2 by 2 Jordan blocks

By its dependence on elementary invariant subspace theory, our algorithms and codes work well with real time parameters $t \in \mathbb{R}$, as well as with more general complex parameters $t \in \mathbb{C}$ as shown in Figs. 1–5.

The web depository [20] also contains a simpler algorithm (`deccompl.m`) for finding the block-diagonal dimensions of a general matrix flow for just one parameter transition from t_a to t_b from the MATLAB `eig` diagonalization of $A(t_a)$ when $t_b \neq t_a$.

The *Matrixflow Decomp* folder at [20] contains these MATLAB m-files for general 1-parameter matrix flows in the subfolder *general flows*. The subfolder *hermitean flows* at [20] deals with hermitean or symmetric single-parameter matrix flows. The hermitean flow methods `decherm.m`, `dechermflow9.m`, and `dechermflow9a.m` are made simpler by the fact that they do not have to deal with the MATLAB `eig.m` m-file output for derogatory non-normal matrices with proper Jordan block structures. There is also a subfolder *staticMatrdecomp* in [20] with a block decomposition code for decomposable general static matrices $A \in \mathbb{C}_{n,n}$. For general non-hermitean matrix flows the actual re-sorting from (**B**) to (**C**) `spy` matrices uses both the zero and the non-zero pattern of each not-all-1s row of a `spy` graph in column (**B**) to arrive at the 0–1 `spy` graph in column (**C**). This helps us

detect the block-diagonal dimensions correctly while also allowing us to determine the total sum of all Jordan block dimensions for such flows.

Each of our MATLAB codes provides on-screen interpretations of the computed outputs and describes the resulting diagonal block dimensions, for both hermitean and general complex matrix flows. For the latter, the on-screen block dimensions refer to the summed dimensions of all Jordan blocks in the listed flow dimensions if followed by a (J). On-screen, there are warnings when the norm of an intermediate matrix $A(t...)$ becomes excessively large in which case the computed block dimension results may be erroneous or unreliable.

Our matrix flow decomposition algorithms take very little time on a 2019 MacBook Pro, around 0.04 sec for matrix flows $A(t)_{n,n}$ of sizes up to 250 by 250 and about 0.9 sec when $n = 1000$ if we do not require plotting. Overall, each algorithm requires one eigenanalysis of one flow matrix $A(t_a)$, two matrix multiplications to obtain $\tilde{V}(...) \cdot A(t_{...}) \cdot \tilde{V}(...)$, and a few steps of logic 0–1 matrix arithmetic which typically looks like the following set of code lines in MATLAB.

```
An = V1' *A2*V1;                % form unitary similarity with V1 for A2
con2 = norm(A2) *10^-13; % standard setting
Lnf = abs(An) >  con2;  % form logic 0-1 matrix of An, threshold = con2
kk = [];   lkk = [];        % sort Lnf into 0-1 row pattern groups :
while length(kk) < n      % start of logic code block
   kd = setdiff(1:n,kk);
   I = kd(1);
   R = Lnf(kd(1),:);
   for i = kd(2:end)
      Ri = R + Lnf(i,:);
      if ismember(2,Ri)
         I = [I,i];
         R = logical(Ri);
      end
   end
   kk = [kk,I];              % accumulate block dimension results
   lkk = [lkk,length(I)];
end                              % end of logic arithmetic code
nn = length(lkk);          % nn is the number of diagonal blocks in An
```

The exceptional Jordan block all-ones rows in our 0–1 `spy` pattern recognition is more involved.

The depository [20] includes MATLAB codes for constructing more than a dozen example flows in both, the hermitean and the general case. Test problem generation can be implemented inside our respective flow decomposing routines by entering an integer matrix dimension number $n \leq 20$. Plotting can be turned off by setting the input parameter `zeich` unequal to 1 and the block dimension information will still be displayed on-screen. Without graphing, the CPU times for running the algorithms were in the hundreds of a second in all tested dimensions and—with graphing included—the computations and all visual '`spy`' displays would appear in a fraction of a second.

The use and success of our matrix flow decomposition algorithm does not depend on or require any smoothness conditions for any given 1-parameter matrix flow $A(t)$. Real-time discontinuous data flow from sensor inputs is quite admissible. Figure 6 below shows the block pattern output for a general complex matrix flow $A(t)$, constructed from a block diagonal time-varying matrix flow $B(t)$ whose blocks have uniform block sizes for all parameters t. In $B(t)$, some diagonal blocks have mixed time-varying and random number entries that change erratically with every call. The matrix flow $B(t)$ is then made dense to become $A(t) = V^* B(t) V$ by using a randomized but fixed unitary matrix similarity V, This creates a dense test example with partially random entries that is decomposable.

In [19], the general complex matrix flow case was generally found to be intractable via coalescing eigencurve studies since general complex matrices usually have complex eigenvalues. In the hermitean matrix flow case, however, all matrix eigenvalues are real and therefore eigencurves can be plotted in terms of the time parameter t in the real plane. Their crossings are easy to spot, report, interpret, and compute. But in the complex matrix flow situation any depiction of eigencurves must involve $\mathbb{R}^3$ for the time parameter t and the real and imaginary parts of the respective eigenvalues.

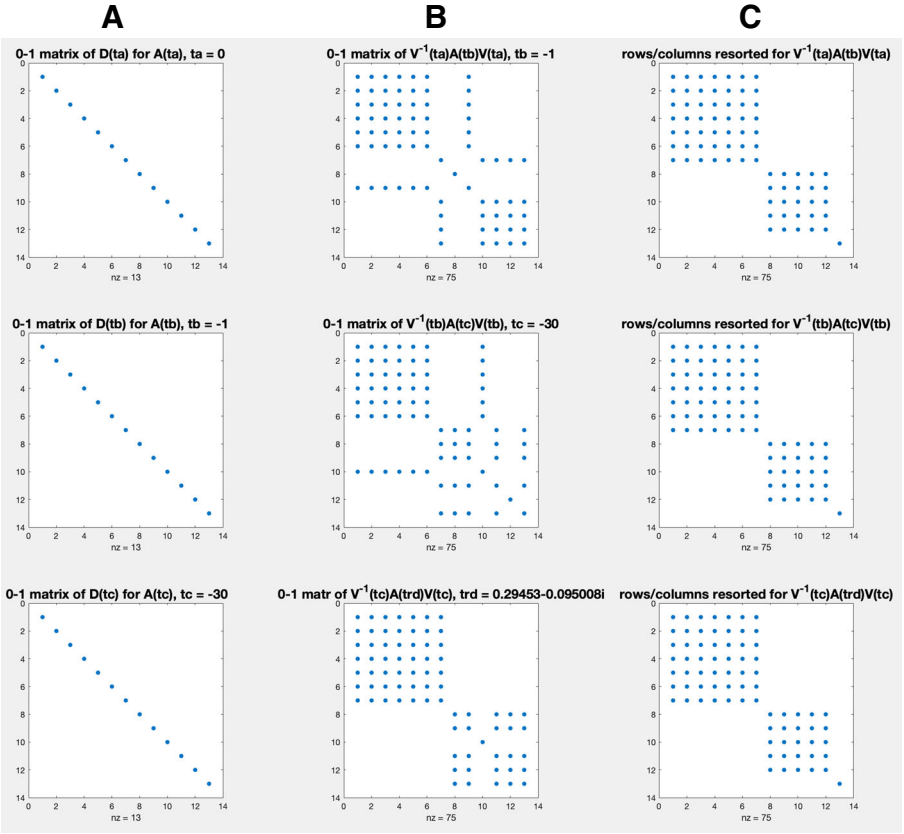


Fig. 6 Decomposable complex flow $A(t)$ with 7-, 5-, and 1-dimensional partially random diagonal blocks

Note that in the invariant subspace based method of this paper this problem does not even appear.

And besides, our new method gives us explicit block-diagonal representations of block-diagonalizable general complex matrix flows and it does so much more expediently than the eigencurve plotting method of [19].

3.3 Computing unitary block-decompositions for complex decomposable matrices

The aim of this paper is to establish that unitary block-decompositions of matrix flows and individual matrices can be computed directly whenever they exist and not depend on the symptomatic indirect methods of eigencurve crossing analyses. According to Theorem 3, the decomposition algorithms for matrix flows extend to individual matrices A by studying the decomposability of the associated 'field of values' hermitean flows $\mathcal{F}_A(t_a) = \cos(t_a)H + \sin(t_a)K$ with $H = (A + A^*)/2$ and $K = (A - A^*)/(2i)$.

Unitarily block-diagonalizable test matrices A can easily be constructed from block-diagonal matrices B after they have been rendered dense by a random entry unitary matrix V in $A = V^*BV$. Non-hermitean but normal matrices such as the 'hanowa' matrix in MATLAB's gallery are good test examples, too. And so are the 15 MATLAB non-normal gallery matrices that are listed as test matrices for their difficult and hard to compute eigen structures. Amazingly, four of these n by n matrices can be unitarily block-diagonalized; some into 2 by 2 blocks, others to 4 by 4 and smaller sized blocks, and some to essentially two $n/2$ by $n/2$ diagonal blocks.

For $n = 196$ e.g., the MATLAB command `decompstaticMatrwspread (gallery('hanowa',196),0)` first checks that the generalized tridiagonal 'hanowa' matrix is indeed normal. Our invariant subspace based algorithm then computes 'hanowa's unitarily equivalent diagonal matrix form via the 'field of values' hermitean matrix flow $\mathcal{F}_A(t)$ in 0.33 s. The computed diagonal entries of the block-reduced 'hanowa' matrix in $\mathbb{R}_{196,196}$ agree with MATLAB's `eig` QR algorithm's computed eigenvalues in their leading 15 digits throughout. Along the way, the spread of entry magnitudes in the block-reduced form for 'hanowa' lies between 10^{-61} and 10^2. This huge increase in matrix entry magnitude spreads of the block reduced 'hanowa' matrix from the modest magnitude spreads between 0.4 and 90 of the hermitean $\mathcal{F}_a(t)$ matrices will reoccur with other MATLAB's gallery matrices and deserves further study.

The tridiagonal normal Kac matrix [17] differs from the 'hanowa' matrix. It is also block diagonalized by our invariant subspace method and it behaves very similarly to 'hanowa' with an entry magnitude spread in its block-diagonal form between 10^{-58} and 10^4 for dimension $n = 196$.

The normal 'circul' matrix in MATLAB's gallery is also diagonalized by our method and its QR method computed eigenvalues agree with our diagonal matrix entries in 16 leading digits. But its entry magnitude spread for $n = 196$ varies only between 10^{-16} and 10^4.

The non-normal 49 by 49 Redheffer matrix 'redheff' in MATLAB's gallery is block diagonalized by our method into ten blocks of sizes 49 by 49, 2 by 2, 2 by

2, and seven times 1 by 1. Its theoretically known 43-fold eigenvalue 1 is poorly computed by both MATLAB's `eig` and from our block-diagonal representation. The magnitude spread of our block diagonal version is again modest of order 10^{18}.

The tridiagonal non-normal 'clement' matrix $A_{n,n}$ in MATLAB's gallery always block-diagonalizes into two blocks of dimensions `floor`$((n+1)/2$ and `floor`$(n/2)$ with modest entry magnitude spreads for $n < 100$ and our method fails to block-diagonalize A when $n >> 100$ due to the standard threshold settings.

The involutory integer 'binomial' n by n matrix in MATLAB's gallery is block-diagonalized into 4 by 4 and smaller sized diagonal blocks for $n \leq 30$ with standard magnitude spreads between 10^{-14} or 10^{-11} and 10^{2} or 10^{6}. For $n = 50$, the lower magnitude limit has increased to 10^{4} and the upper one has grown to 10^{13}. This is uncommon and seems due to the leading blocks in the block diagonal form of 'binomial' with $n = 50$ containing entries of magnitudes 10^{14}, 10^{13}, and 10^{13}, at which time the eigenvalue accuracy has also decreased when compared with MATLAB's `eig` output. This makes our algorithm unreliable for the 'binomial' matrices beyond dimension ≈ 35.

MATLAB's gallery matrix 'invol' is an involutory diagonally scaled Hilbert matrix and it behaves similarly to 'binomial'. It is block-diagonalized into all 2 by 2 blocks by our algorithm for $n \leq 20$, except for an additional 1 by 1 block in odd matrix dimensions. At $n = 20$, the block-diagonalized matrix 'invol' has top diagonal 2 by 2 blocks with entry magnitudes around 10^{15} and 10^{12} and all its off block-diagonal entries have magnitudes around 10^{-1} and 10^{-3}.

The MATLAB code `decompstaticMatrwspread.m` for static matrix block-decompositions and the construction code `Kac.m` for Kac matrices are in the subfolder *staticMatrdecomp* of [20]. The first code requires one input, a general static matrix $A \in \mathbb{C}_{n,n}$ and a second input whether to plot or not to plot 0-1 `spy` plots. It has three outputs, a unitarily similar block-diagonal matrix Ad of A if A is decomposable, the unitary transforming matrix Vc, and a list of the block dimensions of Ad if A was decomposed. On-screen comments explain the results. A run with MATLAB gallery matrices that were exemplified above with small dimensions $n \leq 30$ takes around 0.25 s of CPU time and calls with larger n such as $n = 200$ (when feasible) take around 0.34 s on a 2019 MacBook Pro.

4 Applications

When used in a 'preconditioner' type of way, any matrix block decomposition algorithm ($\mathcal{MBDA}$) may be of benefit in many matrix computational problems, both for fixed entry matrices $A \in \mathbb{C}_{n,n}$ and for time-varying matrix flows $A(t) \in \mathbb{C}_{n,n}$. In particular, if an $\mathcal{MBDA}$ is applied to a similarity invariant or unitarily invariant matrix problem and the given matrix A or matrix flow $A(t)$ is decomposable, then the CPU time savings for computing the desired properties of A or $A(t) \in \mathbb{C}_{n,n}$ amount to around 50% of CPU time if the largest diagonal block has size $m = 0.8 \cdot n$ after $\mathcal{MBDA}$. And if the largest diagonal block dimension after an $\mathcal{MBDA}$ reduction has size $m = n/2$ such as the 'clement' matrix has under our block-diagonalization method, then the savings would reduce the original CPU time by at least three quarters. Here,

we assume that an $O(n^3)$ process is being used on the diagonal blocks in a 'divide and conquer' algorithm after an $\mathcal{MBDA}$ preconditioning. These savings rates have been established in [21]. In [21], the field of values problem for a fixed entry matrix A was studied in light of A's decomposability and our elementary block diagonalization method was applied to the hermitean matrix flow $\mathcal{F}_A(t) = \cos(t)H + \sin(t)K$ with $H = (A + A^*)/2$ and $K = (A - A^*)/(2i)$. This achieved speed-ups of up to 12 times for previously unusable path following methods when compared with the standard QR based matrix eigen-algorithm for computing the field of values boundary curve accurately for decomposable matrices A; again see [21].

The overall cost of finding the eigenvalues for a general dense, but decomposable matrix A is essentially reduced by our algorithm to the cost of one hermitean n by n matrix eigenanalysis plus several smaller sized general matrix block eigenanalyses with a very small additional logic matrix overhead cost.

In Section 3.3, we have found several non-normal matrix classes in MATLAB's gallery list that are unitarily similar to 2 by 2 or 4 by 4 and smaller block diagonal matrices, as well as one dense static n by n matrix type that can be decomposed into exactly two almost equal sized diagonal blocks for all values of n. The concept of block diagonalizability gives rise to a new concept here; it allows us to call the standard class of normal matrices A with $A^*A = AA^*$ 1–normal since they can be unitarily reduced to 1-dimensional block diagonal form. Generalizing the standard general normal matrix concept to k by k blocks, we define k-normal matrices.

Definition 1 A matrix $A \in \mathbb{C}_{n,n}$ is called **k**–normal if A can be unitarily diagonalized into block-diagonal form with maximal block size k by k and additional k by k or smaller dimensioned diagonal blocks.

The `'invol'` test matrices have been shown to be 2–normal for dimensions $2 \leq n \leq 14$. MATLAB's 'binomial' matrices can be decomposed by unitary similarity into four by four and smaller sized diagonal blocks; hence, they are 4–normal. The MATLAB gallery 'clement' matrix can be unitarily block reduced to two almost equal sized diagonal blocks for any dimension n. This makes n by n `'clement'` matrices `floor`$((n+1)/2)$–normal. And the Redheffer matrix was unitarily block-reduced only very slightly from being indecomposable. Finally, the non-hermitean `'circul'` matrices in MATLAB's 'gallery' were diagonalized by our $\mathcal{MBDA}$, making the `'circul'` matrices 1–normal.

We do not know if the block-diagonal reducibility of the above specified matrices via unitary similarities is known, nor whether this has ever been exploited. It is well known how to exploit standard 1–normality of static matrices in theory and computations. Are there efficient methods for 2–normal, 3–normal or 4–normal static matrices?

5 An outlook and adjacent areas of research

It is possible to size each occurring Jordan block in a general matrix flow $A(t)$ individually in Fig. 5 in column (**B**) rather than summarily, but we have not done

so in our $\mathcal{MBDA}$ code. Regarding Jordan structures of fixed static matrices $A_{n,n}$, it appears to be nearly impossible in general and at best very expensive to try and determine the Jordan structure of even small dimensioned static matrices A reliably by numerical means. More involved computational efforts than Francis QR to find the Jordan normal form of small static matrices $A_{n,n}$ reliably are in [18] and similar efforts for the Kronecker normal form of singular matrix pencils are in [9]. Yet the problem of block-diagonalizing time-varying general matrix flows $A(t)$ in the presence of Jordan structures or of repeated eigenvalues has been easily answered computationally here by using elementary invariant subspace theory. This indicates that time-varying or single parameter-varying matrix flows $A(t)$ follow different fundamental concepts than classic static matrix theory and classic matrix numerical analysis.

Applying our matrix flow decomposition method helps with the unitarily invariant matrix field of values problem of decomposing general matrices $A_{n,n} \in \mathbb{C}_{n,n}$, see [21]. The matrix flow decomposition algorithm now allows all path following methods to compute other unitarily invariant matrix problems more efficiently than our more standard global matrix solvers can if the given matrix or matrix flow can be unitarily block-decomposed.

The literature contains several studies that are adjacent to ours, but quite different from actually solving the uniform matrix block-decomposition problem.

Baveley and Stewart [1] construct an algorithm over the reals which finds non-orthogonal matrix similarities with low condition numbers that diagonal block-reduce a real matrix $A_{n,n}$. They start with a unitary similarity on A that reduces the matrix to upper Hessenberg form with (near) zeros on the subdiagonal and with isolated non-zeros that account for complex conjugate eigenvalues of A. This first step is accomplished with a standard QR type orthogonal reduction of A to Hessenberg form without splitting off convergent eigendata. The resulting upper Hessenberg form H is generally dense. Next non-orthogonal Schur complement type eliminations of the upper triangle of H are performed with an eye on low condition numbers for each Schur elimination step. For the Frank matrix of order 12, this method finds a block-diagonalizing similarity with condition number 2920 while Francis QR algorithm computes the eigenvector matrix for the diagonalizable Frank matrix with a condition number around 10^8. For the first test problem matrix J in [1, p. 364] with J replaced by a unitarily similar dense matrix, our complex unitary $\mathcal{MBDA}$ diagonalizes the dense version of J for all perturbations $p = 10^{-1}, .., 10^{-18}$. If we replace the 2 by 2 block in J by a Jordan block for the eigenvalue 1, we retrieve this 2 by 2 block in all our reductions, again for all above values of p. The second example in [1, p. 365] uses a test matrix J with one Jordan block for $\lambda = 1$ and additional simple eigenvalues at $1 + p$, 0.3, 0.4, .., 0.8. Again starting from a dense version of J that was generated by a randomized unitary similarity on J, for $p \geq 10^{-11}$ our algorithm finds the proper block-decomposition, while for $p \leq 10^{-12}$ our $\mathcal{MBDA}$ algorithm coagulates the 4 eigenvalues near 1 into one 4-dimensional diagonal block under unitary similarities. For $p = 0$ Bavely and Stewart's similarity block diagonalizing method [1] also fails to separate the Jordan block for $\lambda = 1$ from J's simple eigenvalue 1.

Murota et al. [14] study a different, yet slightly related problem for a finite set of real symmetric matrices $A_1, ..., A_N$ so that $P^T A_1 P, ..., P^T A_N P$ become simultaneously block diagonal for a fixed orthogonal matrix P. Such problems are solved in [14] via semidefinite programming and theoretically analyzed as matrix $*$-algebra problems. If we interpret the A_i as matrix flow realizations and apply our hermitean matrix flow block-diagonalizing algorithm to the $N = 4$ test matrices of [14, p. 129, p. 144, case (1)], we compute the coarsest possible proper block diagonalization for this 7 by 7 symmetric matrix flow example to have diagonal blocks of sizes 6 and 1. This agrees with the published results from matrix $*$-algebra theory in [14, p. 130]. Murota et al. [14] contains additional results on the structure of the diagonal blocks. The identical block-diagonalization data is also computed with our $\mathcal{MBDA}$ method and the $*$-algebra method for cases (2) and (3) on p. 145 of [14]. If we restrict the problem in case (3) to just A_1 and A_2 in our algorithm, we learn that these two symmetric matrices can be orthogonally block diagonalized to blocks of sizes 3, 1, 1, 1, and 1. And if we use only A_2 and A_3 as flow realizations, these two matrices block-reduce to 2, 1, 1, 1, 1, and 1-dimensional block diagonal form via our $\mathcal{MBDA}$. A_4 finally coalesces these finer blocks to the overall best block sizes 6 and 1. Murota et al. [14, Example 3, p. 143, 144] deals with 3 real symmetric matrices A_1, A_2, and $A_3 = (A_1A_2 + A_2A_1)/2$. Here, both A_1, A_2 and A_1, A_2, A_3 admit the same block diagonalization to blocks of sizes 3 and 1 for both Murota et al.'s method and ours.

Specifically adapted codes for the Bavely and Stewart and the Murota et al. examples are available, together with test matrix generation codes in sub-sub folders of [20].

Dieci et al. in [2] through [8], have dealt with several matrix flow problems that are slightly related to our uniform unitary block-diagonalization study of hermitean matrix flows and of general fixed matrices. For general real matrix flows $A(t)$, [3, 4] deal with computing a smooth orthogonal time-varying matrix flow $X(t)$ with $X(t)(X(t))^T = I$ that upper triagularizes such a matrix flow $A(t)$ by orthogonal similarity. Sabuya [16] has dealt with a related problem to classify all matrix flows $A(t)$ that are block-diagonalizable under time-varying similarities $X^{-1}(t) \cdot A(t) \cdot X(t)$ in contrast to our unified fixed unitary $C^{-1} \cdot A(t) \cdot C$ block-diagonal similarities. Furthermore, Dieci et al. have studied multi-parameter flows in [2, 5–8] and their eigencrossings, as well as their singular value crossings using geometric localization and zoom-in optimization methods.

Separately Loisel and Maxwell [13] have developed an initial value ordinary differential equation (IVP ODE) solver to find eigencrossing points of hermitean block-diagonal matrix flows for the field of values boundary computation problem. With limited success, Loisel and Maxwell have studied three types of troublesome decomposable matrices, see [13, Thm 2.5, Sect 5, 6.2, and 7.1]. We read in [13, Algorithm 6.1 and p. 1733 - 1743]: 'Our algorithm does not apply to type 2 matrices, but this did not have a negative impact for our numerical experiments. We were also not able to completely analyze non-normal matrices of type 3 (when $\lambda_{max}(t)$ can be nonsimple)'. This gave me the impetus to look at eigencurve crossings geometrically

in [19]. The eigencurve detour ultimately let me solve the problem of matrix flow decomposability via constant unitary matrix similarities and construct the application to fixed matrix unitary block-diagonalizations. These insights into a century old open problem were nurtured by my recent time-varying matrix flow and Zhang Neural Network studies and have helped me solve the quantum physics problem of old. We no longer have to rely on detecting matrix decomposability's symptoms through eigencurve studies and we can now check the quantum physics matrix flow models (and others) directly for unitary decomposability.

Acknowledgements I thank the referees and editors who helped with the paper and shared their ideas and knowledge with me.

References

1. Bavely, C.A., Stewart, G.W.: An algorithm for computing reducing subspaces by block-diagonalization. SIAM J. Numer. Anal. **16**, 359–367 (1979)
2. Dieci, L., Eirola, T.: On smooth decompositions of matrices. SIAM J. Matrix Anal. Appl. **20**, 800–819 (1999)
3. Dieci, L., Friedman, M.J.: Continuation of invariant subspaces. Numer. Linear Algebra Appl. **8**, 317–327 (2001). https://doi.org/10.1002/nla.245
4. Dieci, L., Papini, A.: Continuation of eigen-decompositions. Futur. Gener. Comput. Syst. **19**, 1125–1137 (2003)
5. Dieci, L., Pugliese, A.: Singular values of two-parameter matrices: an algorithm to accurately to find their intersections. Math. Comput. Simul. **79**, 1255–1269 (2008)
6. Dieci, L., Gasparo Grazia, M., Papini, A., Pugliese, A.: Locating coalescing singular values of large two-parameter matrices. Math. Comput. Simul. **81**, 996–1005 (2011)
7. Dieci, L., Pugliese, A.: Hermitian matrices depending on three parameters: coalescing eigenvalues. Linear Algebra Appl. **436**, 4120–4142 (2012)
8. Dieci, L., Papini, A., Pugliese, A.: Approximating coalescing points for eigenvalues of Hermitian matrices of three parameters. SIAM J. Matrix Anal. Appl. **34**, 519–541 (2013). [MR 3054590], https://doi.org/10.1137/120898036
9. Embree, M., Keeler, B.: Pseudospectra of matrix pencils for transient analysis of differential-algebraic equations. SIAM J. Matrix Anal. Appl. **38**, 1028–1054 (2017)
10. Hund, F.H.: Zur Deutung der Molekelspektren. I. Z. Phys. **40**, 742–764 (1927)
11. Johnson, C.R.: Numerical determination of the field of values of a general complex matrix. SIAM J. Numer. Anal. **15**, 595–602 (1978). https://doi.org/10.1137/0715039
12. Latter, R.: Atomic energy levels for the Thomas-Fermi and thomas-feIIIli-dirac potential. Phys. Rev. **99**, 510–519 (1955)
13. Loisel, S., Maxwell, P.: Path-following method to determine the field of values of a matrix at high accuracy. SIAM J. Matrix Anal. Appl. **39**, 1726–1749 (2018). https://doi.org/10.1137/17M1148608
14. Murota, K., Kanno, Y., Kojima, M., Kojima, S.: A numerical algorithm for block-diagonal decomposition of matrix – algebras with application to semidefinite programming. Japan J. Indust. Appl. Math. **27**, 125–160 (2010). https://doi.org/10.1007/s13160-010-0006-9
15. von Neumann, J., Wigner, E.P.: On the behavior of the eigenvalues of adiabatic processes. Phys. Z. **30**, 467–470 (1929). reprinted in Quantum Chemistry, Classic Scientific Papers, Hinne Hettema (editor), World Scientific (2000), p. 25–31
16. Sibuya, Y.: Some global properties of matrices of functions of one variable. Math. Ann. **161**, 67–77 (1965)
17. Taussky, O., Todd, J.: Another look at the matrix of Marc Kac. Lin. Alg. Appl. **150**, 341–360 (1991)
18. Trefethen, L.N., Embree, M.: Spectra and pseudospectra: the behavior of nonnormal matrices and operators. Princeton University Press, ISBN 9780691119465, 624 (2005)
19. Uhlig, F.: Coalescing eigenvalues and crossing eigencurves of 1-parameter matrix flows. SIAM J Matrix Anal. Appl. **41**, 1528–1545 (2020). https://doi.org/10.1137/19M1286141

20. Uhlig, F.: The MATLAB codes for plotting and assessing matrix and matrix flow block-diagonalizations are available at http://webhome.auburn.edu/~uhligfd/m_files/MatrixflowDecomp/
21. Uhlig, F.: Constructing the Field of Values of Decomposable and General Square Matrices, submitted, 12 p. arXiv:2006.01241
22. Wilkinson, J.H.: Rounding Errors in Algebraic Processes, [first edition 1963]. Dover Publ. ISBN 978-0486679990, 161 (1994)

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.